

ANNEXURE A: SCENARIO BASED ASSIGNMENT

Course Name: Artificial Intelligence

NAME: ROHIT L

REG NO: 192511236

1. AI-POWERED DRONES FOR EMERGENCY MEDICINE DELIVERY

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region with dynamic obstacles, variable weather/terrain costs, and partial real-time updates.

i. Greedy Best-First Search Behavior, Strengths & Weaknesses:

Greedy Best-First Search uses only the heuristic value $h(n)$ (straight-line distance) to select the next node. *Strengths:* Extremely fast decision-making with low computational overhead, ideal for quick initial route planning in urgent scenarios. *Weaknesses:* Ignores cost already incurred $g(n)$ and actual terrain/weather hazards. Under uncertainty, it easily gets trapped in dead ends or chooses highly risky, blocked paths that look deceptively close to the goal.

ii. A* Handling & Cost/Heuristic Balance:

A* evaluates nodes using $f(n) = g(n) + h(n)$, where $g(n)$ represents path cost (weather delay, terrain risk) and $h(n)$ represents estimated remaining distance. It balances actual past penalties with future estimates, ensuring it avoids deceptively short but high-risk routes.

iii. Impact of Heuristic Accuracy:

If $h(n)$ is admissible and consistent, A* guarantees the optimal safe route. An inaccurate heuristic (overestimating due to flooding or underestimating severe weather) degrades performance: overestimation loses optimality, while severe underestimation degrades A* into slow Uniform Cost Search.

iv. Impact of Dynamic Changes (Blocked Paths):

When roads/airways are suddenly blocked, static paths fail. The search tree must be dynamically updated or re-evaluated, incurring computation delays if recalculated from scratch.

v. Recommendation & Justification:

Recommended Algorithm: Lifelong Planning A* (LPA*) / D* Lite. These incremental heuristic search algorithms reuse previous search trees to adapt to real-time blocked path updates efficiently without full recalculation, offering optimal path guarantees, real-time response, and mission safety.

2. SMART CITY TRAFFIC SIGNAL OPTIMIZATION

Scenario: Optimization of signal timings across hundreds of interconnected intersections to minimize wait time, fuel usage, and congestion under dynamic traffic flows.

Problem Formulation & Traditional Search Inefficiency:

Formulated as an Optimization Problem where State = vector of signal green/red timings across intersections, and Objective Function = minimize total delay + emissions. Traditional search (BFS/DFS/A*) fails because the state space is continuous, high-dimensional, and exponentially large (D^N), requiring exhaustive graph traversal that cannot run in real-time.

i. Hill Climbing Operation & Limitations:

Hill Climbing starts with an initial signal schedule and iteratively changes junction timings to find immediate local improvements. *Limitations:* It is greedy, lacks memory, and fails to explore global possibilities.

ii. Real-World Interpretation of Drawbacks:

Local Maxima: A signal configuration where small timing tweaks increase traffic, yet the overall city grid remains heavily congested. *Plateaus:* Flat regions where small signal timing changes make zero measurable difference in traffic flow. *Ridges:* Sequences of local maxima where optimizing one intersection severely disrupts adjacent intersections.

iii. Simulated Annealing / Genetic Algorithms:

Simulated Annealing allows temporary sub-optimal (bad) signal moves with a decreasing probability (temperature parameter) to escape local bottlenecks. *Genetic Algorithms* maintain a population of signal schedules, combining successful timing patterns via crossover and mutation to find global optima.

iv. Recommendation & Justification:

Recommended Approach: Genetic Algorithms / Reinforcement Learning (MARL). Genetic Algorithms handle vast continuous search spaces and multi-objective optimization across interconnected grids smoothly without stalling at local maxima.

3. AUTONOMOUS MARS ROVER EXPLORATION

Scenario: Mars rover exploring unknown terrain, collecting samples, avoiding hazards under limited sensing and incomplete maps.

i. Online vs Offline Search Necessity:

Offline search requires a complete, static map prior to execution. Because Mars terrain is unknown and unmapped, the rover must interleave computation with physical movement—making Online Search strictly necessary.

ii. Partial Observability & Dynamic Challenges:

The rover's sensors have finite range and suffer from noise (dust storms, shadows). Unseen hazards (loose soil, steep drops) can appear suddenly during movement, invalidating planned trajectories.

iii. Building & Updating Internal Models:

The rover maintains a probabilistic belief state (e.g., Occupancy Grid Map / SLAM). As sensors record new depth and terrain data, Bayesian updates refine hazard probabilities and local map confidence.

iv. Comparison with Uninformed & Informed Search:

Uninformed (BFS/DFS) and Informed (A*) search plan full paths before acting (offline). Online search plans locally, executes step-by-step, learns from real outcome state transitions, and backtracks physically if necessary.

v. Recommendation & Justification:

Recommended Approach: LRTA* (Learning Real-Time A*). It guarantees goal discovery in finite state spaces by updating heuristic estimates of visited states, preventing infinite loops while adapting dynamically to unmapped hazards.

4. AUTOMATED UNIVERSITY EXAM TIMETABLING

Scenario: Automated exam scheduling satisfying student, room, subject order, faculty, and accommodation constraints.

i. CSP Formulation (Variables, Domains, Constraints):

Variables (\$V\$): Exam courses $E_1, E_2, \dots, E_n$.

Domains (\$D\$): Valid time slots and hall combinations (T_i, H_j) .

Hard Constraints: No student assigned to two simultaneous exams; hall capacity $\geq$ enrolled students; prerequisite subject timing sequence ($E_A < E_B$).

Soft Constraints: Maximize spread between difficult exams; faculty preferred slots.

ii. Backtracking Search Operation:

Assigns time/room slots to courses one by one. If a constraint is violated, it backtracks to the previous course assignment and attempts an alternative domain value.

iii. Constraint Propagation (Forward Checking / AC-3):

Forward checking removes illegal slots from unassigned variables' domains immediately after each assignment. AC-3 maintains arc-consistency across the entire constraint graph, pruning huge subtrees before exploration.

iv. Real-World Challenges & Best Scalable Strategy:

Challenges: High combinatorial growth (D^N), tight room limits. **Best Strategy:** Backtracking Search integrated with MRV (Minimum Remaining Values) heuristic, Degree Heuristic, and AC-3 constraint propagation.

5. STRATEGIC GAME AI FOR REAL-TIME STRATEGY (RTS) DECISION MAKING

Scenario: Real-time AI opponent making quick moves in a complex game tree under strict time limits.

i. Minimax Problem Modeling:

Models game states as a tree where AI (MAX) maximizes utility and human opponent (MIN) minimizes AI's utility over alternate turns.

ii. Computational Challenges:

Large branching factor b and depth d result in $O(b^d)$ time complexity, making full depth exploration impossible within real-time millisecond constraints.

iii. Alpha-Beta Pruning Efficiency:

Prunes branches when $\alpha \geq \beta$ (where α is MAX's best score and β is MIN's best score). It reduces best-case time complexity to $O(b^{d/2})$, effectively doubling search depth in equal time.

iv. Evaluation Function Design:

$Eval(s) = w_1 \cdot \text{Material} + w_2 \cdot \text{Territory Control} + w_3 \cdot \text{Resource Gathering Rate} - w_4 \cdot \text{Unit Damage Taken}$. Weights prioritize resource management early-game and army strength late-game.

v. Strategy Recommendation for Real-Time Performance:

Recommended Approach: Depth-Limited Minimax with Alpha-Beta Pruning, Iterative Deepening, Move Ordering, and Transposition Tables. Iterative deepening guarantees a valid, move-ready answer when time expires.